Prescripto AI 2.0 — Database Design & Data Modeling Dossier

Document ID: `DOC-06-DB`
Classification: Academic Engineering Dossier
Canonical Spec Reference: backend/prescripto/db/models/, docs/DATABASE-DESIGN.md
Date of Extraction: November 4, 2026

---

1. Database Technology & Connection Configuration

- **Database Engine:** PostgreSQL 16 (`postgres:16-alpine` on Docker Compose / Render Managed PostgreSQL).
- **ORM & Tooling:** SQLAlchemy 2.0.35 + Alembic 1.20.0 for schema versioning.
- **Driver:** `psycopg2-binary` (v2.9.10).
- **Connection Pool Optimization (Render Free-Tier Guard):**
- `pool_size = 10`
- `max_overflow = 20`
- `pool_pre_ping = True` (Tests liveness before handing connections to requests)
- `pool_recycle = 1800` (Recycles idle connections every 30 minutes to prevent drops by cloud load balancers)
- `pool_timeout = 30` (Seconds before throwing connection acquisition timeout)
- **Local Development Driver:** SQLite 3 with `check_same_thread = False` for hermetic unit testing.

---

2. Complete Entity-Relationship (ER) Diagram

```
erDiagram
    USERS ||--o{ PRESCRIPTION_DOCUMENTS : "uploads"
    USERS ||--o{ REVIEWS : "conducts"
    USERS ||--o{ DELETION_JOBS : "requests"

    PATIENTS ||--o{ PRESCRIPTION_DOCUMENTS : "associates"

    PRESCRIPTION_DOCUMENTS ||--|| ANALYSES : "triggers"
    PRESCRIPTION_DOCUMENTS ||--o{ DELETION_JOBS : "targets"

    ANALYSES ||--|| ANALYSIS_JOBS : "queued as"
    ANALYSES ||--o{ ANALYSIS_STAGES : "executes"
    ANALYSES ||--o{ PRESCRIPTION_MEDICATIONS : "extracts"
    ANALYSES ||--o{ RISK_FINDINGS : "flags"
    ANALYSES ||--o{ REVIEWS : "receives"

    PRESCRIPTION_MEDICATIONS ||--o{ MEDICATION_CANDIDATES : "evaluates"
    MEDICATIONS ||--o{ MEDICATION_CANDIDATES : "resolves to"

    USERS {
        uuid id PK
        string username UK
        string email UK
        string hashed_password
        string role
        boolean is_active
        datetime created_at
    }

    PATIENTS {
        uuid id PK
        string patient_id UK
        string name
        int age
        string gender
        string phone_number
        string allergies
        text medical_history
        datetime created_at
    }

    PRESCRIPTION_DOCUMENTS {
        uuid id PK
        uuid patient_id FK
        uuid uploaded_by_user_id FK
        string patient_ref
        string storage_key
        string file_hash_sha256
```

```
        string mime_type
        int file_size_bytes
        string status
        datetime created_at
    }

    ANALYSES {
        uuid id PK
        uuid document_id FK
        string status
        boolean review_required
        jsonb review_reasons
        datetime created_at
        datetime completed_at
    }

    ANALYSIS_JOBS {
        uuid id PK
        uuid analysis_id FK
        string status
        string worker_id
        string lease_token
        int retry_count
        datetime heartbeat_at
        datetime expires_at
        datetime created_at
    }

    ANALYSIS_STAGES {
        uuid id PK
        uuid analysis_id FK
        string stage_name
        string status
        jsonb stage_metadata
        datetime started_at
        datetime completed_at
    }

    MEDICATIONS {
        uuid id PK
        string brand_name
        string generic_name
        jsonb active_ingredients
        string strength
        string dosage_form
        string route
        string rxnorm_cui
        string source_class
        string source_version
        string verification_status
        jsonb provenance_metadata
```

```
    }

    PRESCRIPTION_MEDICATIONS {
        uuid id PK
        uuid analysis_id FK
        int line_index
        string name_raw
        string name_state
        string strength_raw
        string strength_state
        string dose_raw
        string dose_state
        string unit_raw
        string unit_state
        string route_raw
        string route_state
        string frequency_raw
        string frequency_state
        string duration_raw
        string duration_state
        string instructions_raw
        string instructions_state
    }

    MEDICATION_CANDIDATES {
        uuid id PK
        uuid prescription_med_id FK
        uuid analysis_id FK
        string extracted_text
        string matching_strategy
        float candidate_score
        uuid resolved_medication_id FK
        string resolution_status
    }

    RISK_FINDINGS {
        uuid id PK
        uuid analysis_id FK
        string check_type
        string finding_key
        string finding_status
        jsonb medication_ids
        text evidence_text
        string source_name
        float confidence_score
        string not_evaluated_reason
    }

    REVIEWS {
        uuid id PK
        uuid analysis_id FK
```

```
        uuid reviewer_id FK
        string action
        text notes
        datetime signed_at
    }

    IDEMPOTENCY_RECORDS {
        string idempotency_key PK
        uuid prescription_id
        uuid analysis_id
        string payload_hash
        int response_status_code
        datetime created_at
    }

    DELETION_JOBS {
        uuid id PK
        uuid document_id FK
        string status
        uuid requested_by_user_id FK
        boolean manifest_written
        string manifest_hash
        datetime requested_at
        datetime completed_at
    }

    TOKEN_BLOCKLIST {
        uuid id PK
        string jti UK
        string token_type
        datetime revoked_at
        datetime expires_at
    }
```

---

3. Detailed Table Schema Definitions

3.1. `users` Table

Stores system identity credentials and clinical roles. * `id` (`UUID`, Primary Key, default `gen_random_uuid()`) * `username` (`VARCHAR(64)`, Unique, Non-nullable, Indexed) * `email` (`VARCHAR(255)`, Unique, Non-nullable, Indexed) * `hashed_password` (`VARCHAR(255)`, Non-nullable): Salted bcrypt password hash. * `role` (`VARCHAR(32)`, Non-nullable): Enum `OPERATOR`, `REVIEWER`, `ADMIN`, `PATIENT`. * `is_active` (`BOOLEAN`, Non-nullable, default `True`) * `created_at` (`TIMESTAMPTZ`, Non-nullable, default `NOW()`)

3.2. `patients` Table

Records de-identified patient demographic and clinical history. * `id` (`UUID`, Primary Key) * `patient_id` (`VARCHAR(64)`, Unique, Non-nullable, Indexed): External clinic MRN/ID. * `name` (`VARCHAR(255)`, Non-nullable) * `age` (`INTEGER`, Nullable) * `gender` (`VARCHAR(16)`, Nullable): `MALE`, `FEMALE`, `OTHER`. * `phone_number` (`VARCHAR(32)`, Nullable, Indexed): Used for patient OTP matching. * `allergies` (`TEXT`,

Nullable): Comma/line-separated known drug allergen tokens. * `medical_history` (`TEXT`, Nullable): Chronic conditions (e.g., Hypertension, Diabetes). * `created_at` (`TIMESTAMPTZ`, Non-nullable, default `NOW()`)

3.3. `prescription_documents` Table

Represents an ingested prescription document artifact. * `id` (`UUID`, Primary Key) * `patient_id` (`UUID`, Nullable, Foreign Key → `patients.id`) * `uploaded_by_user_id` (`UUID`, Non-nullable, Foreign Key → `users.id`) * `patient_ref` (`VARCHAR(64)`, Nullable): Correlation identifier. * `storage_key` (`VARCHAR(512)`, Non-nullable): S3 object key. * `file_hash_sha256` (`CHAR(64)`, Non-nullable): Binary payload cryptographic hash. * `mime_type` (`VARCHAR(64)`, Non-nullable): `image/jpeg`, `image/png`, `image/tiff`. * `file_size_bytes` (`INTEGER`, Non-nullable): Validated $\le 20\text{ MB}$. * `status` (`VARCHAR(32)`, Non-nullable): Enum `RECEIVED`, `STORED`, `DELETED`. * `created_at` (`TIMESTAMPTZ`, Non-nullable, default `NOW()`)

3.4. `analyses` Table

Master clinical analysis record tracking pipeline state and review status. * `id` (`UUID`, Primary Key) * `document_id` (`UUID`, Non-nullable, Foreign Key → `prescription_documents.id`, Unique) * `status` (`VARCHAR(32)`, Non-nullable): `PENDING`, `PROCESSING`, `REQUIRES_REVIEW`, `COMPLETED`, `FAILED_STALLED`, `REVIEWED_COMPLETE`. * `review_required` (`BOOLEAN`, Non-nullable, default `False`) * `review_reasons` (`JSONB`, Nullable): Array of triggered review reason strings. * `created_at` (`TIMESTAMPTZ`, Non-nullable, default `NOW()`) * `completed_at` (`TIMESTAMPTZ`, Nullable)

3.5. `analysis_jobs` Table (The Asynchronous Queue)

High-throughput row-level queue queried via `FOR UPDATE SKIP LOCKED`. * `id` (`UUID`, Primary Key) * `analysis_id` (`UUID`, Non-nullable, Foreign Key → `analyses.id`, Unique) * `status` (`VARCHAR(32)`, Non-nullable, Indexed): `QUEUED`, `RUNNING`, `COMPLETED`, `FAILED`. * `worker_id` (`VARCHAR(64)`, Nullable): Hostname + PID of the processing worker. * `lease_token` (`VARCHAR(64)`, Nullable): Monotonic generation fencing token. * `retry_count` (`INTEGER`, Non-nullable, default `0`) * `heartbeat_at` (`TIMESTAMPTZ`, Nullable) * `expires_at` (`TIMESTAMPTZ`, Nullable, Indexed): Timestamp when worker lease lapses. * `created_at` (`TIMESTAMPTZ`, Non-nullable, default `NOW()`)

3.6. `medications` Table (Formulary Master)

CDSCO verified pharmacological drug catalogue. * `id` (`UUID`, Primary Key) * `brand_name` (`VARCHAR(255)`, Non-nullable, Indexed) * `generic_name` (`VARCHAR(255)`, Non-nullable, Indexed) * `active_ingredients` (`JSONB` / `ARRAY`, Non-nullable): Array of chemical strings. * `strength` (`VARCHAR(64)`, Nullable) * `dosage_form` (`VARCHAR(64)`, Nullable): `Tablet`, `Capsule`, `Syrup`, `Injection`. * `route` (`VARCHAR(64)`, Nullable): `Oral`, `Topical`, `Inhalation`, `Ophthalmic`. * `rxnorm_cui` (`VARCHAR(32)`, Nullable): Standardized RxNorm CUI concept identifier. * `source_class` (`VARCHAR(64)`, Non-nullable): `CDSCO_REGULATORY`. * `source_version` (`VARCHAR(32)`, Non-nullable, default `"2024.1"`) * `verification_status` (`VARCHAR(32)`, Non-nullable): `VERIFIED_AUTHORITY`. * `provenance_metadata` (`JSONB`, Nullable)

3.7. `prescription_medications` Table

Structured clinical extractions preserving field-level uncertainty. * `id` (`UUID`, Primary Key) * `analysis_id` (`UUID`, Non-nullable, Foreign Key → `analyses.id`, Indexed) * `line_index` (`INTEGER`, Non-nullable) * 8 Structured Fields each with raw string, uncertainty state, and confidence: * `name_raw`, `name_state` (`CLEAR` / `AMBIGUOUS` / `UNREADABLE` / `NOT_PRESENT`), `name_confidence` * `strength_raw`, `strength_state`, `strength_confidence` * `dose_raw`, `dose_state`, `dose_confidence` * `unit_raw`, `unit_state`, `unit_confidence` * `route_raw`, `route_state`, `route_confidence` * `frequency_raw`, `frequency_state`,

`frequency_confidence` * `duration_raw`, `duration_state`, `duration_confidence` * `instructions_raw`, `instructions_state`, `instructions_confidence`

3.8. `risk_findings` Table

Clinical safety warnings generated by deterministic domain engines. * `id` (`UUID`, Primary Key) * `analysis_id` (`UUID`, Non-nullable, Foreign Key → `analyses.id`, Indexed) * `check_type` (`VARCHAR(64)`, Non-nullable): `DUPLICATE_MEDICATION`, `ADVERSE_EFFECT`, `ALLERGY_CONFLICT`. * `finding_key` (`VARCHAR(128)`, Non-nullable): Unique hash preventing duplicate findings. * `finding_status` (`VARCHAR(32)`, Non-nullable): `POTENTIAL`, `REQUIRES_REVIEW`, `NOT_EVALUATED`. * `medication_ids` (`JSONB` / `ARRAY`, Non-nullable): Foreign IDs of implicated medicine lines. * `evidence_text` (`TEXT`, Non-nullable): Human-readable clinical rationale. * `source_name` (`VARCHAR(64)`, Non-nullable): `PRESCRIPTO_DETERMINISTIC_RULES`. * `confidence_score` (`FLOAT`, Nullable) * `not_evaluated_reason` (`VARCHAR(64)`, Nullable): `UNRESOLVED_IDENTITY`, `AMBIGUOUS_IDENTITY`.

---

4. Alembic Migration History

The schema evolution is tracked under backend/prescripto/db/migrations/versions/:

1. **`0001_initial_schema.py`** (`rev = '0001'`):
2. Establishes core enum types: `fieldstate`, `resolutionstatus`, `findingstatus`, `safetychecktype`, `analysisstatus`, `jobstatus`, `documentstatus`, `role`.
3. Creates initial tables: `users`, `prescription_documents`, `analyses`, `analysis_jobs`, `analysis_stages`, `medications`, `prescription_medications`, `medication_candidates`, `risk_findings`, `reviews`, `model_versions`, `token_blocklist`, `deletion_jobs`, `audit_events`.
4. **`0002_idempotency_records.py`** (`rev = '0002'`, down-revision `'0001'`):
5. Creates `idempotency_records` table to support HTTP `Idempotency-Key` tracking and prevent upload replay race conditions.
6. **`0003_add_patients_table.py`** (`rev = '0003'`, down-revision `'0002'`):
7. Creates `patients` table with unique constraint on `patient_id`.
8. Safely adds `patient_id` foreign key column to `prescription_documents`.
9. Conditionally updates PostgreSQL `role` enum type to include `'PATIENT'`.